Task 2: Git Reset vs. Git Revert

Step-by-step lab solution with commands, expected observations, and comparison tables

Objective: Understand how `git reset` and `git revert` undo commits, and how the `--soft`, `--mixed`, and `--hard` options affect the staging area and working directory.

Part 1 — Create a repository and make three application commits

Step 1: Create a new repository and the initial commit

```
mkdir GitResetRevert
cd GitResetRevert
git init -b main

echo "# Git Reset and Revert Lab" > README.md
git add README.md
git commit -m "Initial commit"
```

This creates a repository on the `main` branch and records the initial commit.

Step 2: Create app.py and make the second commit

```
echo "print('Version 1')" > app.py
git add app.py
git commit -m "Add app.py"
```

The file `app.py` now contains `print('Version 1')`.

Step 3: Make two additional commits

```
echo "print('Version 2')" > app.py
git add app.py
git commit -m "Update app to version 2"

echo "print('Version 3')" > app.py
git add app.py
git commit -m "Update app to version 3"

git log --oneline
```

Example history (commit hashes will differ):

```
d4e5f6a Update app to version 3
c3d4e5f Update app to version 2
b2c3d4e Add app.py
a1b2c3d Initial commit
```

There are four commits in total: the initial commit and three commits involving the application file.

Part 2 — Undo the latest commit using git reset

Step 4: Use the default reset behavior

```
git reset HEAD~1
git status
git log --oneline
```

This is equivalent to `git reset --mixed HEAD~1`. The latest commit is removed from the current branch history, but its changes remain in the working directory as unstaged changes.

Expected status:

```
On branch main
Changes not staged for commit:
  modified: app.py
```

The file still contains `print('Version 3')`. The branch pointer moved back to the previous commit, while the file modification remains in the working directory.

Part 3 — Undo the latest commit using git revert

Step 5: Restore the latest commit for a fair comparison

```
git add app.py
git commit -m "Update app to version 3"
git log --oneline
```

This recreates the latest application commit so all four original commits are present again.

Step 6: Revert the latest commit

```
git revert --no-edit HEAD
git status
git log --oneline
```

Example history:

```
e5f6a7b Revert "Update app to version 3"
d4e5f6a Update app to version 3
c3d4e5f Update app to version 2
b2c3d4e Add app.py
a1b2c3d Initial commit
```

The file `app.py` now contains `print('Version 2')`. The original commit remains in history, and a new commit reverses its changes. If the revert succeeds and there are no other changes, `git status` reports a clean working tree.

Part 4 — Compare reset and revert

Feature	git reset HEAD~1	git revert HEAD
Original latest commit	Removed from current branch history	Retained in history
New commit created	No	Yes
Working directory	Default mixed reset leaves changes unstaged	Contains the reversed changes
Typical status	Modified, unstaged file	Clean after successful revert if no other changes
Best suited for	Reorganizing local, unpublished history	Undoing changes safely in shared history

Key difference: `git reset` moves the current branch pointer backward. `git revert` creates a new commit that reverses an earlier commit.

Part 5 — Explore the three git reset flags

For each experiment below, begin from the original four-commit state with `app.py` containing `print('Version 3')`. Run each experiment separately from that same starting state. If you use the reset commands one after another without restoring the starting state, the results will differ.

Experiment A: --soft

```
git reset --soft HEAD~1
git status
git log --oneline
```

Observation: The latest commit is removed from the current branch history. Its changes remain staged, and the working directory still contains Version 3.

```
Changes to be committed:
  modified: app.py
```

Meaning: The commit is undone, but its changes remain staged and ready for another commit.

Experiment B: --mixed

```
git reset --mixed HEAD~1
git status
git log --oneline
```

Observation: The latest commit is removed from the current branch history. Its changes are unstaged but remain in the working directory.

```
Changes not staged for commit:
  modified: app.py
```

Meaning: This is the default behavior of `git reset`.

Experiment C: --hard

```
git reset --hard HEAD~1
git status
git log --oneline
```

Observation: The latest commit is removed from the current branch history. The staging area and tracked files in the working directory are updated to match the target commit. `app.py` now contains `print('Version 2')`.

```
On branch main
nothing to commit, working tree clean
```

Warning: `git reset --hard` discards uncommitted changes to tracked files. Such changes may be difficult to recover.

Final summary

Command	Commit history	Staging area	Working directory
reset --soft HEAD~1	Latest commit removed	Changes remain staged	Changes remain
reset --mixed HEAD~1	Latest commit removed	Changes unstaged	Changes remain
reset --hard HEAD~1	Latest commit removed	Matches target commit	Matches target commit
revert HEAD	Original retained; new revert commit added	Normally clean after completion	Contains reversed changes

Remember: Soft = keep changes staged. Mixed = keep changes unstaged. Hard = make tracked files match the target commit. Revert = create a new commit that undoes an earlier commit.